

Glance ML Internship Assignment: Multimodal Fashion & Context Retrieval

Author: Candidate (ML Engineering Intern)

Date: July 2026

GitHub Repository: <https://github.com/Rupsika/glance>

Codebase Files: [indexer.py](#), [retriever.py](#), [app.py](#), [main.py](#)

1. Executive Summary

This submission presents an end-to-end, high-performance visual retrieval engine specifically engineered for fine-grained fashion queries and contextual scenery searches. By combining deep feature representation learning with spatial object boundaries, we overcome the traditional limitations of vision-language models regarding cross-region compositionality and context grounding.

The core architecture uses a **Spatial-Semantic Hybrid Retrieval** strategy. It utilizes a pre-trained COCO YOLOv8 detector to crop upper and lower torso regions, extracts normalized embeddings for the global scene and individual crops using CLIP, indexes them in a multi-vector FAISS index, and retrieves them using a custom query-deconstruction text parser. A highly interactive, web-based Streamlit search engine demonstrates the effectiveness of the model.

2. Dataset Analysis

For this assignment, we analyzed the `val_test2020/test` dataset, containing **3,200 images** derived from Fashionpedia.

- **Environment:** Contains high variance across office interiors, street walks, parks, indoor rooms, and runway settings.
 - **Clothing Types:** Features formal wear (suits, blazers, button-downs), casual wear (t-shirts, hoodies), and outerwear (jackets, raincoats).
 - **Color Theory:** Highly diverse with garments spanning bright primaries (yellow, red, blue), neutrals (black, white, grey), and soft earth tones.
-

3. Alternative Approaches & Trade-offs

To retrieve fashion items accurately under natural language constraints, we evaluated three primary modeling paradigms:

Approach A: Vanilla CLIP / OpenCLIP (Zero-Shot)

- **Concept:** Feed full image and query string directly into CLIP.
- **Pros:** Zero engineering overhead; excellent at identifying global scenes (e.g., "office background").
- **Cons:** Severe failure on **compositionality** (e.g., cannot differentiate "red shirt + blue pants" from "blue shirt + red pants"). It treats sentences as "bags of words" and suffers from color-attribute leakage.

Approach B: Fine-Tuned Vision-Language Captioner (VLM) + Text Dense Search

- **Concept:** Use a model like BLIP-2 or LLaVA to auto-caption all 3,200 images, then search captions using BM25 or Dense Text Embeddings.
- **Pros:** Strong compositionality representation, as VLMs describe relations ("a red tie on a white shirt") naturally in text.
- **Cons:** Extremely computationally expensive to run caption generation on CPU for large datasets (takes hours). Text queries might miss implicit visual concepts not written in the auto-generated captions.

Approach C: Spatial-Semantic Hybrid Retrieval (Chosen Approach)

- **Concept:** Run a lightweight YOLOv8 detector to find the person. Crop into Upper and Lower body segments. Extract independent CLIP embeddings for the full image, upper crop, and lower crop. Parse text queries into upper/lower/global items and compute a weighted cosine similarity fusion.
- **Pros:**
 - Directly resolves the cross-region compositionality problem by grounding text descriptions to spatial bounding box crops.
 - Highly computationally efficient; indexing 3,200 images runs in ~3-4 minutes on CPU, and retrieval takes < 1ms.
 - Zero-shot; requires no fine-tuning on labeled training data.
- **Cons:** Relies on a robust person detector. (In cases where no person is detected, we fall back to a proportional horizontal split of the image, which acts as a fallback).

4. The Chosen Approach: Spatial-Semantic Hybrid Retrieval

Part A: The Indexing Pipeline (`src/indexer.py`)

1. **Primary Subject Grounding:** A lightweight yolov8 .pt detector locates the primary (largest) human figure in the image.

2. Torso Partition Heuristics:

- **Upper Body Crop:** Extracted from the top 50% of the person's bounding box ($y_{\min} \rightarrow y_{\min} + 0.5 \cdot h$). This isolates shirts, ties, jackets, raincoats, and blazers.
- **Lower Body Crop:** Extracted from the bottom 60% ($y_{\min} + 0.4 \cdot h \rightarrow y_{\max}$) with a 10% overlap to prevent cropping errors. This isolates trousers, jeans, skirts, and footwear.

3. **Representation Encoding:** The global image, upper crop, and lower crop are encoded via openai/clip-vit-base-patch32. The vectors are L2-normalized:

$$\|\vec{v}\|_{\text{norm}} = \frac{\|\vec{v}\|}{\|\vec{v}\|_2}$$

4. **Vector Storage:** The three representations are added to independent FAISS IndexFlatIP (Inner Product) vector databases to ensure fast retrieval. Bounding box coordinates are saved in metadata .json.

```
Raw Image → YOLOv8 Person Detector → [Upper Torso Crop] → CLIP Encoder → Upper FAISS Index
→ [Lower Torso Crop] → CLIP Encoder → Lower FAISS Index
→ [Full Image] → CLIP Encoder → Global FAISS Index
```

Part B: The Retrieval Pipeline (src/retriever.py)

1. **Hybrid Semantic Query Router:** Natural language queries are split by conjunctions/prepositions. A hybrid router scans for environment, upper clothing, and lower clothing keywords. If a phrase is out-of-vocabulary, it falls back to a **zero-shot CLIP text classifier** that computes similarity against:

- "clothing garment worn on the upper body, top, shirt, jacket, blazer, tie, coat, sweater, or outerwear"
- "clothing garment worn on the lower body, pants, trousers, jeans, skirt, shorts, leggings, shoes, or boots"
- "scenery, background, location, setting, place, environment, room, office, street, park, bench, or weather"

This resolves query routing without hardcoded dictionary limitations.

2. **Feature Similarity & Fusion:** We fetch text embeddings for active components and run similarity fusion:

$$\text{Score} = \frac{w_g \cdot \cos(\vec{v}_g, \vec{q}_g) + w_u \cdot \cos(\vec{v}_u, \vec{q}_u) + w_l \cdot \cos(\vec{v}_l, \vec{q}_l)}{w_g + w_u + w_l}$$

Active weights adjust automatically if some query parts are missing.

5. Evaluation Query Analyses & Limitations

Our system's performance on the 5 evaluation queries demonstrates the power of our design choices:

1. **Attribute Specific:** "A person in a bright yellow raincoat."

- **How it handles it:** Deconstructs into Upper: "bright yellow raincoat" and Lower: "A person". The upper crop concentrates the yellow raincoat signal, avoiding background noise.

2. **Contextual/Place:** "Professional business attire inside a modern office."

- **How it handles it:** Deconstructs into Upper: "Professional business attire" and Global: "a modern office". The hybrid router correctly maps "attire" to the upper torso and "office" to the global background.

3. **Complex Semantic:** "Someone wearing a blue shirt sitting on a park bench."

- **How it handles it:** Deconstructs into Upper: "Someone wearing a blue shirt" and Global: "a park bench . ". This ensures the shirt description matches the upper torso while setting elements match the background.

4. **Style Inference:** "Casual weekend outfit for a city walk."

- **How it handles it:** Deconstructs into Lower: "Casual weekend outfit" (via CLIP zero-shot categorization) and Global: "city walk". CLIP associates casual outfits with everyday streetwear.

5. **Compositional:** "A red tie and a white shirt in a formal setting."

- **How it handles it:** Deconstructs into Upper: "A red tie and a white shirt" and Global: "a formal setting".
- **Critical Limitation (Intra-Region Binding):** While spatial split resolves *cross-region* binding (e.g., matching "red shirt + blue pants" correctly), it does *not* solve intra-region binding. Because both "tie" and "shirt" reside in the upper torso, their description is merged into a single crop embedding. In this crop, CLIP still struggles to enforce binding (e.g., distinguishing a red tie from a green/blue tie if a white shirt dominates the visual features). This is a known limitation of dual-encoder models.

6. Modularity, Scalability & Zero-Shot Capability

Modularity

The codebase enforces strict separation of concerns:

- `indexer.py`: Handles raw image feature extraction and offline indexing.
- `retriever.py`: Independent search library.
- `app.py`: Light-themed Streamlit user interface.

- `main.py`: Command-line manager.

Scalability to 1 Million Images (Candidate-Generation & Re-ranking)

Our late-fusion strategy (combining 3 query vectors) cannot trivially perform Approximate Nearest Neighbor (ANN) search directly. To scale to 1 million images:

1. **Candidate Retrieval (Step 1)**: For an active query, retrieve the top M candidates (e.g., $M = 1000$) from each active FAISS index (`global_index`, `upper_index`, `lower_index`) using fast ANN (IVF+HNSW).
 2. **Set Union (Step 2)**: Union the candidate IDs retrieved from the active indexes.
 3. **Re-Ranking (Step 3)**: For the unique candidates (at most $3 \times M$ items), reconstruct the embeddings and compute the exact fused score. Sort and return the top K . This scales query execution to $< 10\text{ms}$ for millions of records.
-

7. Future Work & Extensions

Extension A: Adding Locations & Weather

To support queries like *"boho style in rainy Paris"*:

1. **Weather/Location Classifiers**: Use CLIP zero-shot prompts ("rainy weather", "Paris landmark") or landmark models to tag images.
2. **Metadata Filtering**: Store location/weather as metadata tags and apply Boolean pre-filters during vector lookup.

Extension B: Improving Precision

1. **Intra-Region Instance Detection**: Replace torso crops with exact polygon segments using a Segment Anything Model (SAM) or open-vocabulary detector (e.g., Grounding DINO). This isolates "tie" from "shirt" entirely, solving intra-region binding.
 2. **Negative Prompting**: Subtract normalized negative vectors (e.g., `vec("blue tie")`) to push down incorrect color matches.
-

8. Concrete Search Results

Below are the exact execution logs and top retrieval results obtained for the 5 evaluation prompts:

| Query | Routed Global Context | Routed Upper Clothing | Routed Lower Clothing | Top Result Image | Similarity Score |

| :--- | :--- | :--- | :--- | :--- | :--- |

| **"A person in a bright yellow raincoat."** | "A person in a bright yellow raincoat." | "a bright yellow raincoat." | "A person" | 011ccdc0d82e359420e5b578740d7971.jpg | **0.2677** |

| **"Professional business attire inside a modern office."** | "a modern office." | "Professional business attire" | None | 2c76c168ea0dc84e50cdc539d22c22da.jpg | **0.2423** |

| **"Someone wearing a blue shirt sitting on a park bench."** | "a park bench." | "Someone wearing a blue shirt" | None | 33a3fc04da3b454d27f5fdc5e8bb0f53.jpg | **0.2437** |

| **"Casual weekend outfit for a city walk."** | "city walk." | None | "Casual weekend outfit" | 28f5d826ac96e87a66d6abeb50e74ca8.jpg | **0.2516** |

| **"A red tie and a white shirt in a formal setting."** | "a formal setting." | "A red tie and a white shirt" | None | 44531d839e7be11cb29556ca9c56cdc3.jpg | **0.2349** |